

Mini Project – 3

Yoga Pose Detection & Feedback System

Objective: Use the Video Dataset prepared by your seniors that contains short videos of students performing Yogasanas. Ensure the dataset contains diverse backgrounds, ambient lighting and clothes. If not, add these variations. Build a system that can suggest improvements in yoga posture in real-time.

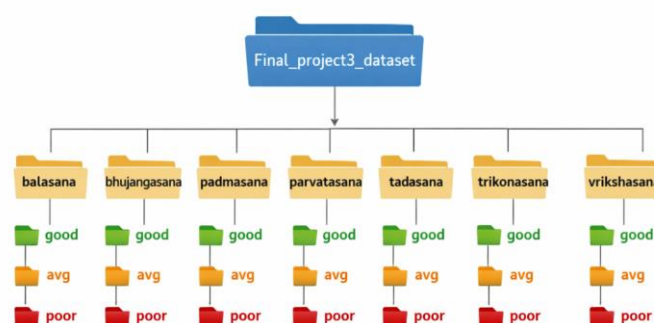
Description of the model:

1. Overview

The model is designed to **classify yoga poses from video data** using body joint information extracted from each frame. It combines **computer vision (MediaPipe)** with a **machine learning classifier (Random Forest)** to recognize different poses.

2. Data Collection

- Dataset consists of **yoga pose videos** organized in folders:



- Each video represents a specific yoga posture performed under different conditions (lighting, background, etc.)

3. Pose Landmark Extraction

- The model uses **MediaPipe Pose** to extract **33 body landmarks** from each frame.
- Each landmark contains:
 - x (horizontal position)
 - y (vertical position)
 - z (depth)

Total features per frame = $33 \times 3 = 99$ features

4. Data Preprocessing

Frame Sampling

- Instead of using every frame, the model processes **every 5th frame**
- Reduces redundancy and improves efficiency

Normalization

- Landmarks are normalized by:
 - Centering at **left hip**
 - Scaling based on maximum coordinate value

This makes the model:

- Independent of **person position**
- Robust to **distance from camera**
- Less sensitive to **body size differences**

5. Feature Dataset Creation

- Extracted landmarks are flattened into a feature vector
- Stored as:
 - **X (features)** → landmark values
 - **y (labels)** → corresponding yoga pose

6. Model Used

The classification model is: **RandomForestClassifier**

Why Random Forest?

- Handles high-dimensional data well (99 features)
- Robust to noise
- Works well without heavy tuning
- Suitable for tabular landmark data

Configuration:

- `n_estimators = 300` → 300 decision trees
- `class_weight = 'balanced'` → handles class imbalance

7. Training Process

- Dataset split:
 - 80% training
 - 20% testing
- Model learns patterns between **body joint positions and pose labels**

8. Model Evaluation

- Metric used: **Accuracy**
- Predictions are made on test data and compared with actual labels

9. Model Saving

- Final trained model is saved using **pickle** as: `yoga_model_2.pkl`
- This allows reuse in real-time applications

Python Implementation:

```
import cv2
import mediapipe as mp
import numpy as np
import joblib

# ✓ Load compressed model
model = joblib.load("yoga_model_2.pkl")

# Initialize MediaPipe
mp_pose = mp.solutions.pose
mp_drawing = mp.solutions.drawing_utils
```

```
pose_model = mp_pose.Pose()

# Angle calculation
def calculate_angle(a, b, c):
    a = np.array(a)
    b = np.array(b)
    c = np.array(c)

    radians = np.arctan2(c[1]-b[1], c[0]-b[0]) - \
               np.arctan2(a[1]-b[1], a[0]-b[0])

    angle = np.abs(radians * 180.0 / np.pi)

    if angle > 180:
        angle = 360 - angle

    return angle

# UPDATED FEATURE EXTRACTION (VERY IMPORTANT)
def extract_features(frame):

    image = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
    results = pose_model.process(image)

    if results.pose_landmarks:

        lm = results.pose_landmarks.landmark

        coords = []
        for point in lm:
            coords.extend([point.x, point.y, point.z])

        coords = np.array(coords).reshape(33, 3)

        # ✓NORMALIZATION
        center = coords[23] # LEFT_HIP
        coords = coords - center

        max_value = np.max(np.abs(coords))
        if max_value != 0:
            coords = coords / max_value

        # ✓ANGLES
        shoulder = coords[11][:2]
        elbow = coords[13][:2]
        wrist = coords[15][:2]
```

```
hip = coords[23][:2]
knee = coords[25][:2]
ankle = coords[27][:2]

elbow_angle = calculate_angle(shoulder, elbow, wrist)
knee_angle = calculate_angle(hip, knee, ankle)

# NEW FEATURES

# 1 Ankle distance
left_ankle = coords[27][:2]
right_ankle = coords[28][:2]
ankle_distance = np.linalg.norm(
    np.array(left_ankle) - np.array(right_ankle)
)

# 2 Knee height difference
left_knee_y = coords[25][1]
right_knee_y = coords[26][1]
knee_height_diff = abs(left_knee_y - right_knee_y)

# 3 Hip alignment
left_hip = coords[23][:2]
right_hip = coords[24][:2]
hip_distance = np.linalg.norm(
    np.array(left_hip) - np.array(right_hip)
)

# ✔ FINAL FEATURE VECTOR
features = coords.flatten().tolist() + [
    elbow_angle,
    knee_angle,
    ankle_distance,
    knee_height_diff,
    hip_distance
]

return features, results.pose_landmarks, elbow_angle

return None, None, None

# Feedback logic
def get_feedback(prediction):

    prediction = prediction.lower()

    pose = prediction.split("_")[0] # e.g. balasana
    quality = prediction.split("_")[-1] # good / avg / poor
```

```
feedback_map = {

    "balasana": {
        "poor": "Relax your hips back toward heels and lower your chest fully.",
        "avg": "Good, but try to extend your arms forward and relax deeper.",
        "good": "Perfect Balasana. Calm and well-aligned."
    },

    "bhujangasana": {
        "poor": "Lift your chest higher and keep elbows slightly bent.",
        "avg": "Good, open your chest more and pull shoulders back.",
        "good": "Excellent Bhujangasana. Strong spinal extension."
    },

    "padmasana": {
        "poor": "Adjust your legs and keep your spine straight.",
        "avg": "Good, but sit more upright and relax your shoulders.",
        "good": "Perfect Padmasana. Stable and relaxed posture."
    },

    "parvatasana": {
        "poor": "Lift your hips higher and straighten your back.",
        "avg": "Good, press your heels slightly towards the ground.",
        "good": "Great Parvatasana. Strong and stable alignment."
    },

    "tadasana": {
        "poor": "Stand straight and distribute weight evenly.",
        "avg": "Good, engage your core and relax shoulders.",
        "good": "Perfect Tadasana. Balanced and aligned."
    },

    "trikonasana": {
        "poor": "Extend your arms fully and avoid bending forward.",
        "avg": "Good, open your chest more and align your torso.",
        "good": "Excellent Trikonasana. Great body alignment."
    },

    "vrikshasan": {
        "poor": "Focus on balance and place your foot firmly.",
        "avg": "Good, stabilize your standing leg and align hips.",
        "good": "Perfect Vrikshasana. Strong balance and posture."
    }
}

# Default fallback (production safety)
```

```
    if pose in feedback_map and quality in feedback_map[pose]:
        return feedback_map[pose][quality]
    else:
        return "Adjust your posture and try again."

# Start webcam
cap = cv2.VideoCapture(0)

# Fullscreen window
cv2.namedWindow("Yoga Posture Detection", cv2.WINDOW_NORMAL)
cv2.setWindowProperty("Yoga Posture Detection",
                      cv2.WND_PROP_FULLSCREEN,
                      cv2.WINDOW_FULLSCREEN)

while cap.isOpened():

    ret, frame = cap.read()
    if not ret:
        break

    features, pose_landmarks, elbow_angle = extract_features(frame)

    if features:

        # Prediction using NEW features
        prediction = model.predict([features])[0]

        feedback = get_feedback(prediction)

        # Display prediction
        cv2.putText(frame, prediction, (30, 50),
                   cv2.FONT_HERSHEY_SIMPLEX, 1.2, (0, 255, 0), 3)

        # Display feedback
        cv2.putText(frame, feedback, (30, 100),
                   cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 0, 255), 2)

        # Display angle
        cv2.putText(frame, f"Elbow Angle: {int(elbow_angle)}",
                   (30, 150),
                   cv2.FONT_HERSHEY_SIMPLEX, 1, (255, 255, 0), 2)

        # Draw skeleton
        mp_drawing.draw_landmarks(
            frame, pose_landmarks, mp_pose.POSE_CONNECTIONS)

    cv2.imshow("Yoga Posture Detection", frame)
```

```
        if cv2.waitKey(10) & 0xFF == ord('q'):  
            break  
  
cap.release()  
cv2.destroyAllWindows()
```

Description of code:

1. Importing Libraries

- cv2 → handles webcam and image processing
- mediapipe → detects human body landmarks
- numpy → numerical operations (arrays, math)
- pickle → loads your trained ML model

2. Loading Trained Model

- Loads the pre-trained **RandomForestClassifier**
- This model predicts: pose_quality → e.g. balasana_good, trikonasana_poor

3. Initializing Pose Detection

Uses **MediaPipe Pose**

Detects **33 body landmarks** in real-time.

4. Angle Calculation Function

```
def calculate_angle(a, b, c):
```

Calculates angle between 3 points (joint angle)

Example:

- Shoulder–Elbow–Wrist → elbow angle
- Hip–Knee–Ankle → knee angle

5. Feature Extraction

Step 1: Convert image

```
image = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
```


- MediaPipe works with RGB, not BGR

Step 2: Detect landmarks

```
results = pose_model.process(image)
```

- Returns 33 body points if a person is detected

Step 3: Extract coordinates

```
coords = []
```

```
for point in lm:
```

```
    coords.extend([point.x, point.y, point.z])
```

- Converts landmarks into a feature list
- Shape becomes:
- $(33 \text{ landmarks} \times 3 \text{ values}) = 99 \text{ features}$

Step 4: Normalization

```
center = coords[23] # LEFT_HIP
```

```
coords = coords - center
```

```
coords = coords / max_value
```

It removes:

- camera distance effect
- body position variation
- Makes model robust and consistent

Step 5: Angle Features

```
elbow_angle = calculate_angle(...)
```

```
knee_angle = calculate_angle(...)
```

Adds biomechanical understanding to the model

Step 6: Extra Custom Features

1. Ankle Distance

```
ankle_distance = np.linalg.norm(left_ankle - right_ankle)
```

Helps detect:

- standing vs wide-leg poses

2. Knee Height Difference

```
knee_height_diff = abs(left_knee_y - right_knee_y)
```

Helps detect:

- asymmetry (important for poses like Trikonasana)

3. Hip Alignment

```
hip_distance = np.linalg.norm(left_hip - right_hip)
```

Helps detect:

- posture stability

Step 7: Final Feature Vector

```
features = coords.flatten().tolist() + [...]
```

Final input to model:

- 99 landmark values
 - angles
 - custom features

Output: return features, pose_landmarks, elbow_angle

6. Feedback System

```
def get_feedback(prediction):
```

How it works:

- Splits prediction: balasana_good → pose=balasana, quality=good
- Returns **custom feedback message**

7. Webcam Initialization

```
cap = cv2.VideoCapture(0)
```

Starts live camera feed

8. Fullscreen Display

```
cv2.namedWindow(...)
```

```
cv2.setWindowProperty(...)
```

Opens output in fullscreen mode

9. Main Loop

```
while cap.isOpened():
```

Step-by-step flow:

1. Capture frame

```
ret, frame = cap.read()
```

2. Extract features

```
features, pose_landmarks, elbow_angle = extract_features(frame)
```

3. Predict pose

```
prediction = model.predict([features])[0]
```

4. Get feedback

```
feedback = get_feedback(prediction)
```

5. Display results

```
cv2.putText(...)
```

Displays:

- Pose name
- Feedback
- Elbow angle

6. Draw skeleton

```
mp_drawing.draw_landmarks(...)
```

- Visualizes body joints

7. Show frame

```
cv2.imshow(...)
```

8. Exit condition

```
if cv2.waitKey(10) & 0xFF == ord('q'):
```

Press **Q** to quit

9. Cleanup

```
cap.release()
```

```
cv2.destroyAllWindows()
```

- Stops camera
- Closes window

Output :











Performance Evaluation :

- The performance of the yoga pose detection system is evaluated based on its ability to **accurately classify yoga poses in real-time** using body landmark features extracted via **MediaPipe Pose** and a trained **RandomForestClassifier**.
- The model achieves good classification accuracy on static yoga poses and performs efficiently in real-time.
 - Measures the percentage of correctly predicted poses
 - Calculated on test dataset
- Formula:
$$\text{Accuracy} = (\text{Correct Predictions} / \text{Total Predictions})$$

Accuracy = 0.8837545126353791